

Contents

Topics	Page No.
Unit-5: Publishing Android Applications	1
Chapter-11: Publishing Android Applications	1
11.0 Structure of Publishing Android Applications	1
11.1 Learning Outcomes	1
11.2 Publishing Android Application	1
11.2.1 Preparing the application for release.....	3
11.2.2 Releasing the android application to the users..	7
11.3 Self-Assessment Questions.....	8
11.4 Self-Assessment Activities.....	9
11.5 Multiple-Choice Questions.....	9
11.6 Key Answers to Multiple-Choice Questions.....	11
11.7 Summary	11
11.8 Keywords.....	12
11.9 Recommended Resources for Further Reading.....	12

Structure of Publishing Android Applications

- 11.1 Learning Outcomes
 - 11.2 Publishing Android Applications
 - 11.3 Self-Assessment Questions
 - 11.4 Self-Assessment Activities
 - 11.5 Multiple-Choice Questions
 - 11.6 Key answers to multiple-choice questions
 - 11.7 Summary
 - 11.8 Keywords
 - 11.9 Recommended resources for further reading
-

11.1 Learning Outcomes

After the successful completion of this chapter, the student will be able to:

- Describe the phases of publishing an android application
 - Describe versioning used in preparing an android application for release
 - Generate a self-signed certificate and digitally sign an android application for release
 - Explain and release an android application to the users
-

11.2 Publishing Android Application

Once an android application is developed and tested, it can be made available to the users. The final phase of android application development is Publishing as shown in figure 11.1. Users can use the application on their devices only after publishing. There are various ways by which the app can be released/made available to users. To release an application to the users, it is necessary to create a .apk file and distribute it to them so that the user can install and run the application on an android device. The Source code (.java file) written in JAVA is converted by the JAVA compiler (javac) into a .class file. The Dex compiler then converts the generated .class file into a .dex file which in turn is later converted into a .apk file by the Android Asset Packaging Tool (AAPT).

If the application is to be distributed on the google play store, the signing process has to be completed. Signing means adding additional private key information to the .apk file.

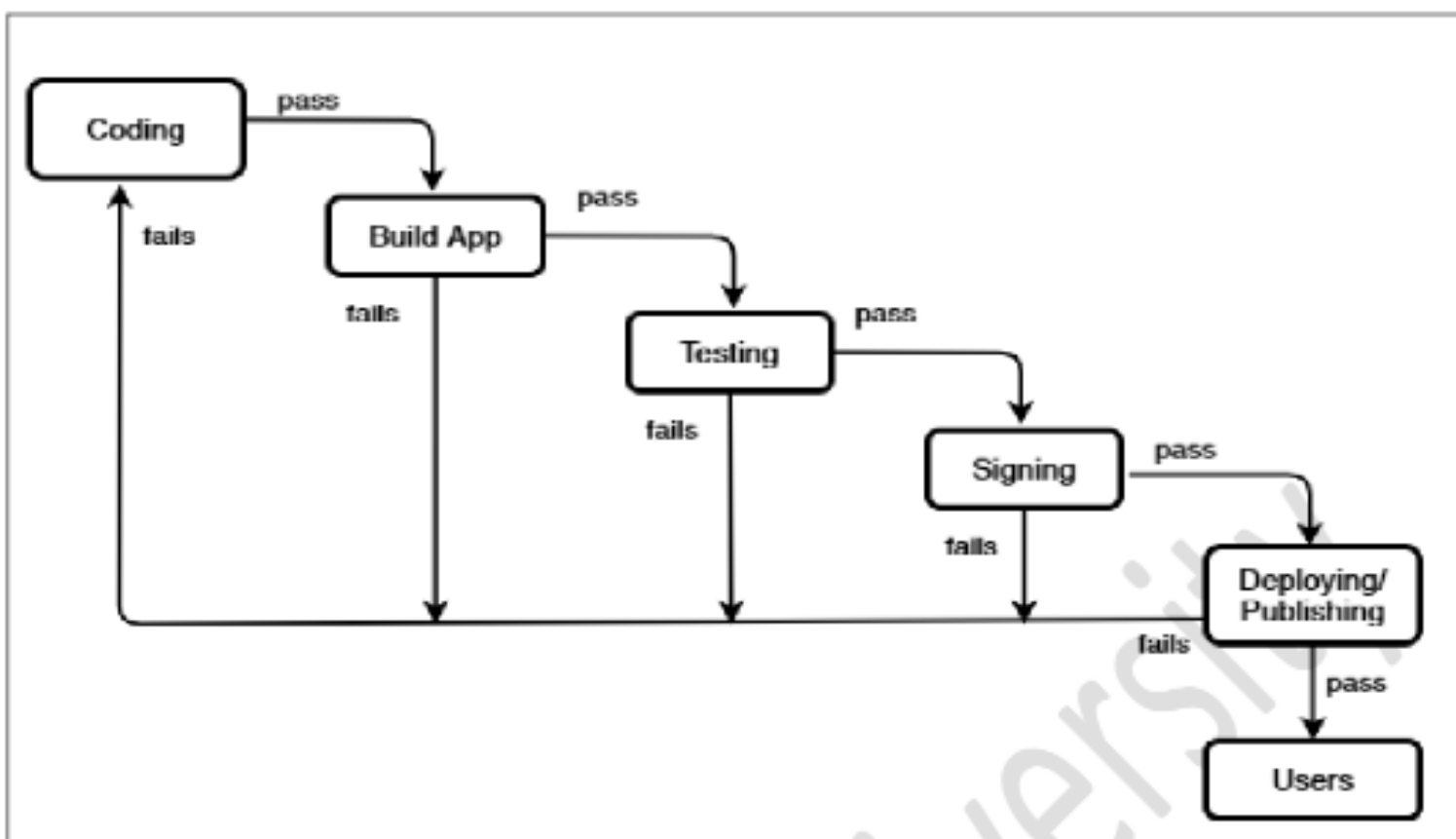


Figure 11.1: Android application development and deployment phases

Process of Publishing an android application

The process of publishing consists of 2 phases :

1. Preparing the application for release
2. Releasing the application to the users

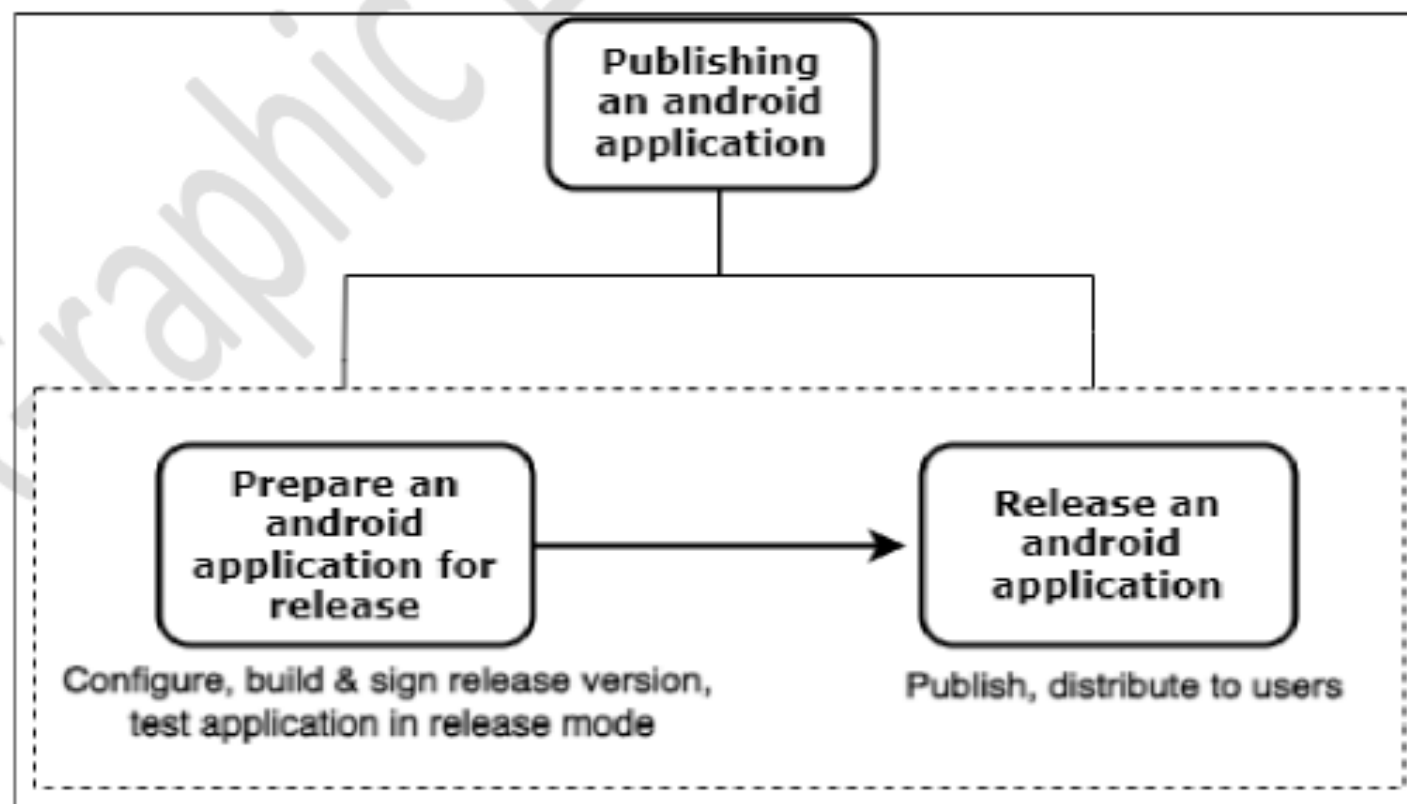


Figure 11.2: Publishing an android application consists of 2 phases

11.2.1 Preparing the application for release

The following tasks have to be performed before the application is released

1. Specify a suitable name and icon for the application
2. Provide proper versioning for the application
3. Remove unwanted resources, garbage code, and any logging calls used
4. Configure the application with the required permissions in the AndroidManifest.xml file
5. Update all the application resources like graphics and multimedia files
6. Prepare remote servers used by the application if any
7. Generate a self-signed certificate, digitally sign the application and build an apk
8. Test the release version

1. Specify a suitable name and icon for the application

Create and use an icon for the application, so that the users can identify and launch the application easily. It helps to manage the application once it is distributed. Choose a suitable name for the application before distribution because the name cannot be changed once it is distributed. These changes have to be made in the AndroidManifest.xml file by setting the values of `android:icon` and `android:label` attributes in the `<application>` element

```
<application>
```

```
    android:icon="@mipmap/xxx"           // specify an icon
```

```
    android:label="@string/app_name"     // application name
```

```
    . . .
```

```
    . . .
```

```
</application>
```

2. Provide proper versioning for the application

Versioning: It plays an important role in the application development life cycle.

- It helps users to identify which version of the release is installed on their device and upgrade if new release versions are available
- It helps to determine the compatibility and dependencies with other applications
- The version information can be used even by publishing services to perform the tasks depending upon the release version

There are 2 attributes used to set the application version

- `versionCode`
- `versionName`

These values are set in the `AndroidManifest.xml` file directly or in the gradle build file which is then merged during the process of build in the manifest file.

versionCode: It is a positive integer value used only by the android system for the internal purpose and is not shown/known to the users. The higher value indicates the latest version. This number is used by the android system to prevent users from installing apk with a lower number than the existing one on the device.

For e.g. `android:versionCode = "1"`

versionName: It is a string value given in the format "<major>.<minor>.<point>".

For e.g. `android:versionName = "1.1.0"`

where the major value is 1, the minor value is 1, and the point value is 0. The value of each of these is incremented by 1 based on the changes made.

<major> value is changed when a new feature/module is added

<minor> value is changed when some small features are added which were rolled out during development

<point> value is changed when some bugs are fixed and were not noticed earlier.

`versionName` represents the application release version.

Remove unwanted resources, garbage code, and any logging calls used
Any log calls used in the application should be removed. The `debuggable` attribute in the `AndroidManifest.xml` file should be set to false `android:debuggable = "false"`. Any unwanted code, display messages, and resources used for testing flow and checking the proper functioning of the application should be removed.

Configure the application with the required permissions in the `AndroidManifest.xml` file

The permissions in the `AndroidManifest.xml` file should be verified.

- To send SMS through the application check whether the permission is set
`<uses-permission android:name="android.permission.SEND_SMS"/>`

- To read/write data stored on external storage verify
`<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>`
`<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>`
- The API levels used in the application are set by using attributes in `<uses-sdk>` element
 - `android:minSdkVersion` – It is used to specify the minimum level of API on which the android application can run. The default value is 1.
 - `android:maxSdkVersion` – It is used to specify the maximum level of API on which the android application can run. It is an optional attribute.
 - `android:targetSdkVersion` – It is used to specify the level of API on which the android application is designed to run.

```
<uses-sdk
    android:minSdkVersion="14"
    android:targetSdkVersion="19"
/ >
```

5. Update all the application resources like graphics and multimedia files
 Update the required graphics and image/icon files in the drawable folder and multimedia files such as audio, and video used by the application
6. Prepare remote servers used by the application if any
 If the android application is using any database stored on remote servers, check the connectivity and ensure the availability of the network so that the application works properly without any network issues.
7. Generate a self-signed certificate, digitally sign the application and build an apk

Any android application has to be digitally signed by the developer before it is released in the market. All android applications require a digital certificate and a private key of that certificate to install/upload the application in Google play. Google does not appoint any authority to sign the certificates, they are self-signed by the developers.

The certificate is used to identify the developer and also share data with other applications that are signed with the same certificate. The same key is used by the developers to publish different or multiple applications. New versions/releases of the applications also use the same certificate for release in the market. Users on the other hand have to fully install the upgraded version if a different certificate is used. Android applications that are not signed are rejected by the emulator or any device.

The signing of an android application is done in 2 modes

- Debug
- Release

Signing in debug mode: In the android studio, any application that is developed is automatically signed in debug mode when the project is run from the IDE. By default, a debug certificate/Keystore with an unknown password and a private key with an unknown password is automatically generated by the android SDK. So whenever the developer makes the changes and runs the project on the emulator or even a real device that is connected using USB to the development machine, it does not ask for a password. However, an android application signed in debug mode cannot be distributed to the users.

Signing in release mode :

To sign a project in release mode, a Keystore and private key are created.

Keystore: It is a binary file that consists of a set of private keys. This file has to be kept in a secure and safe folder.

Private/Upload key: It is used to identify the author/developer of the application

There are 2 ways for generating a signed apk for an android application in android studio

1. On the menu bar, select build and then use Generate signed APK wizard and provide the required information
2. Configure the project to automatically release the signed APK during the build process by providing the required information in the Open Module setting, an option available in the project browser on right click of the app

Standard command line tools are also available which are used to generate keys and sign an android application

For e.g. keytool is used to generate keys for an android application

jarsigner is used to sign an android application

zipalign is used to optimize the .apk file

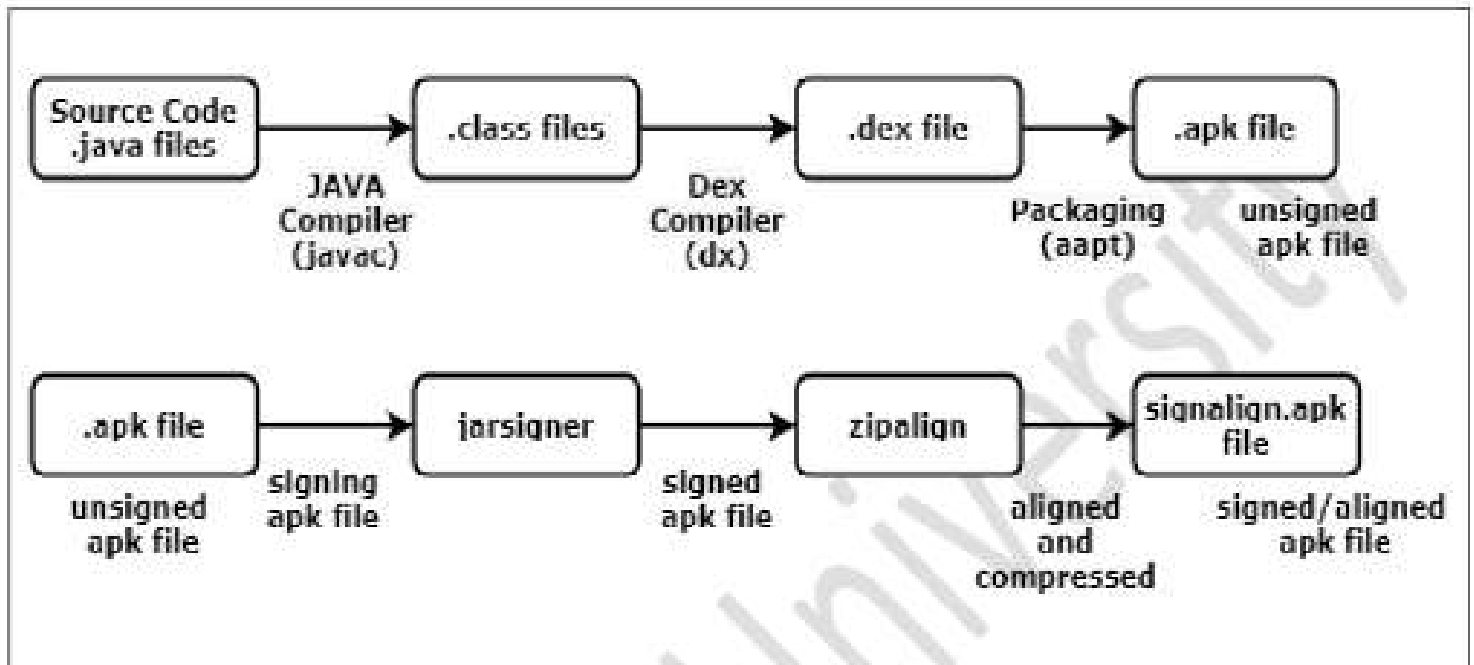


Figure 11.3: Process of converting java source file to signed and aligned .apk file

3. Test the release version

Test the release version of the application on real devices such as handsets, tablets, etc. to ensure that the user interface is user-friendly and working properly on all devices.

2.2 Releasing the android application to the users

Once the android application is prepared for release and tested it is ready to be distributed to the users

There are several ways by which the android application can be released.

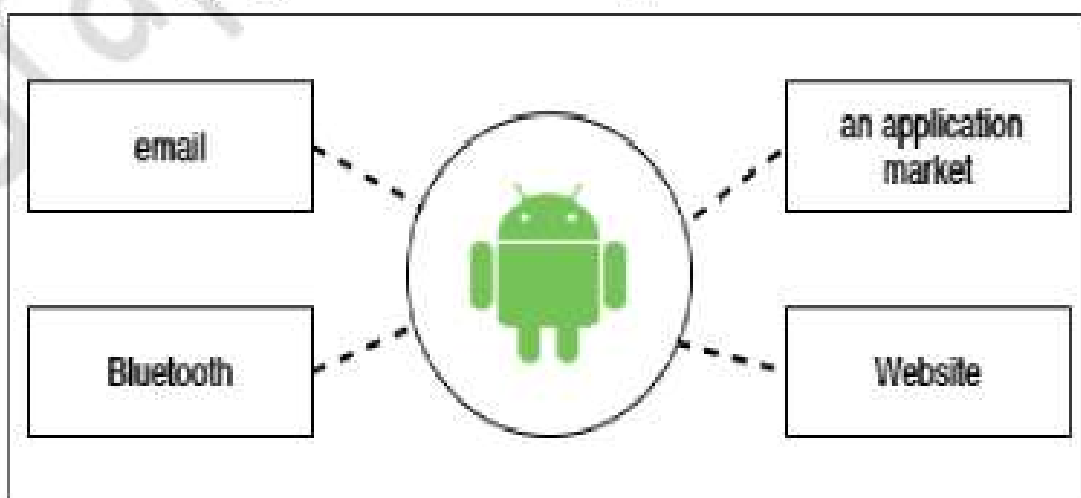


Figure 11.4: Different Ways of releasing android applications

- Email – It is one of the simplest and fastest ways to publish an android application. Email the .apk file to the user. But it is unsafe as anyone can pirate and use it.
- Download from a Website – Host the android application for release on a website so that the user can download and install it. But this is also unsafe as it's difficult to keep a track of the users who are downloading and using it.
- Bluetooth - .apk file can also be transferred to a mobile device by using Bluetooth by pairing it with the device. Only authorized users can be paired and allowed for file transfer.
- Google Play - It is an official marketplace for distributing android applications. Google Play supports publicity, selling, and distribution of android applications not only locally but throughout the world.

To release an application in Google Play

- Create a Google Developer Account by paying registration fees and signing the terms and agreement policies.
- Product description, feedback questionnaire for content rating, and pricing information for the application are provided through the wizards before uploading the .apk files.
- For uploading the app if the app size is more than 100 MB which is the limitation, APK expansion files are used.
- The app can be published by either using alpha, beta, or production options.
- Alpha version [closed testing] is available for testing only for a specific number of invited testers.
- Any interested users, who have been given permission can test the Beta version and send feedback,
- Production release is available to all users of the play store in the countries chosen.

11.3 Self-Assessment Questions

- Q1. List and explain with a neat diagram the phases of publishing an android application.
[8 marks, L2]
- Q2. What are the tasks to be performed while preparing an android application for release?
[8 marks, L2]
- Q3. Explain the different ways of releasing an android application in brief.[8 marks, L2]

- Q4. Explain how to generate a self-signed certificate, digitally sign the application and build an apk. [6 marks, L2]
- Q5. Explain the signing of an android application in brief.[6 marks, L2]
- Q6. Explain the elements which are to be updated while preparing an android application for release.[4 marks, L2]
- Q7. Explain the versioning of an android application in brief.[6 marks, L2]

11.4 Self-Assessment Activities

Note: Use an appropriate emulator for the execution of an android application.

- A1. Create an android application and display the message "My first android application for release".
- Run the application by using an emulator
 - Run the application by using a real device
 - Transfer the .apk file using Bluetooth to a real device and run the application.

11.5 Multiple-Choice Questions

1. The no of phases in publishing an android application is _____. [1 mark, L1]
 - a) 2
 - b) 3
 - c) 4
 - d) 8
2. To release an android application the file created is an _____ file. [1 mark, L1]
 - a) exe
 - b) dll
 - c) apk
 - d) None of the above
3. A suitable name and icon for an android application before release is set in the _____ file. [1 mark, L1]
 - a) acitivity_main.xml
 - b) AndroidManifest.xml
 - c) MainActivity.java
 - d) None of the above

4. The attributes used to set the version of an android application are _____. [1 mark, L1]
- a) appversionCode and appversionName
 - b) appCode and appName
 - c) versionCode and versionName
 - d) None of the above
5. The format of the value used to set the versionName of an android application where "X" is an integer is _____. [1 mark, L1]
- a) "X"
 - b) "X.X.X"
 - c) "X.X"
 - d) None of the above
6. The file that consists of a set of private keys used for releasing an android application is known as _____. [1 mark, L1]
- a) Keyfile
 - b) KeyName
 - c) KeyValues
 - d) Keystore
7. To generate keys for an android application the tool used is _____. [1 mark, L1]
- a) keytool
 - b) Keystore
 - c) jarsigner
 - d) zipalign
8. An android application signed in the _____ mode cannot be distributed to the users. [1 mark, L1]
- a) release
 - b) debug
 - c) private
 - d) public
9. The attribute used to specify the level of API on which the android application is designed to run is _____. [1 mark, L1]
- a) android:minSdkVersion

- b) android:maxSdkVersion
- c) android:targetSdkVersion
- d) None of the above

10. An android application cannot be released using _____. [1 mark, L1]

- a) email
- b) website
- c) Google Play
- d) Google Map

11.6 Key Answers to the multiple-choice question

1. The no of phases in publishing an android application is 2. [a]
2. To release an android application the file created is an apk file. [c]
3. A suitable name and icon for an android application before release is set in the AndroidManifest.xml file. [b]
4. The attributes used to set the version of an android application are versionCode and versionName.[c]
5. The format of the value used to set the versionName of an android application where "X" is an integer is "X.X.X". [b]
6. The file that consists of a set of private keys used for releasing an android application is known as Keystore. [d]
7. To generate keys for an android application the tool used is Keytool. [a]
8. An android application signed in the debug mode cannot be distributed to the users.[b]
9. The attribute used to specify the level of API on which the android application is designed to run is android:targetSdkVersion. [c]
10. An android application cannot be released using Google maps.[d]

11.7 Summary

Once an android application is developed and tested, it can be made available to the users. The final phase of android application development is Publishing. Users can use the application on their devices only after publishing. To release an application to the users, it is necessary to create a .apk file and distribute it to them so that the user can install and run the application on an android device.

The Source code (.java file) written in JAVA is converted by the JAVA compiler (javac) into a .class file. The Dex compiler then converts the generated .class file into a .dex file

which in turn is later converted into a .apk file by the Android Asset Packaging Tool (AAPT).

If the application is to be distributed on the google play store, the signing process has to be completed. Signing means adding additional private key information to the .apk file.

The process of publishing consists of 2 phases :

- Preparing the application for release
- Releasing the application to the users

11.8 Keywords

- Publishing
- Android Asset Packaging Tool (AAPT)
- Android Package Kit (APK)
- Keystore
- Keytool

11.9 Recommended Resources for Further Reading

Text Book(s)

1. Pradeep Kothari, "Android Application Development" Black-Book, Dreamtech Press, 2015
2. Wei-Meng Lee, "Beginning Android Application Development", Wiley 2011.
3. Dawn Griffiths and David Griffiths, "Head First Android Development", O'Reilly, 2017.

References

- <https://www.javatpoint.com/android-tutorial>
- <https://www.tutorialspoint.com/android/index.htm>
- <https://developer.android.com/guide/components/fundamentals>

Phases of Publishing Android Applications

© 2020 Pearson Education, Inc.



Introduction

- Once an android application is developed and tested, it can be made available to the users.
- The final phase of android application development is Publishing.
- Users can use the application on their devices only after publishing.
- There are various ways by which the app can be released/made available to users.

© 2020 Pearson Education, Inc.



Introduction

- To release an application to the users, it is necessary to create a .apk file and distribute it to them so that the user can install and run the application on an android device.
- The Source code (.java file) written in JAVA is converted by the JAVA compiler (javac) into a .class file. The Dex compiler then converts the generated .class file into a .dex file which in turn is later converted into a .apk file by the Android Asset Packaging Tool (AAPT).

© All Rights Reserved



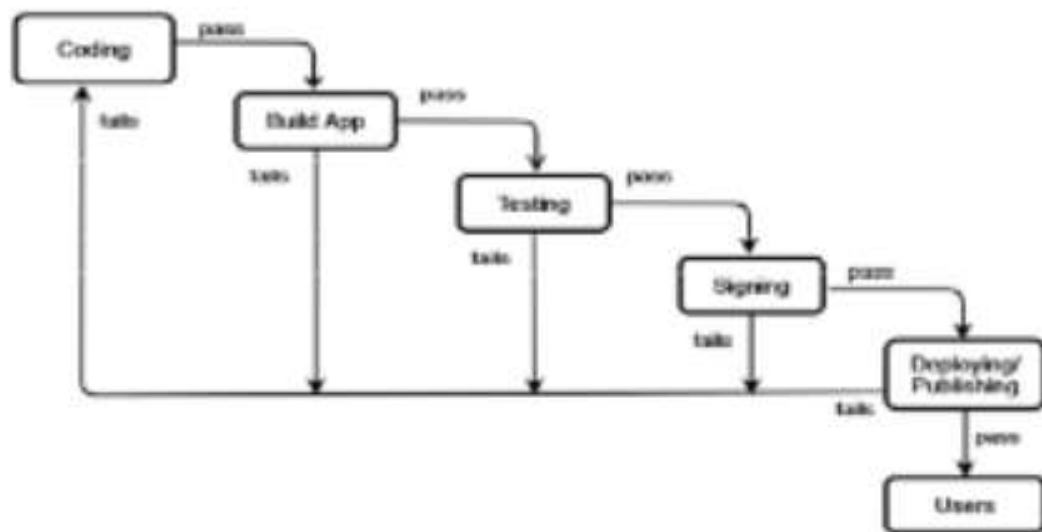
Introduction

- If the application is to be distributed on the google play store, the signing process has to be completed.
- Signing means adding additional private key information to the .apk file.

© All Rights Reserved



Android Application Development and Deployment Phases



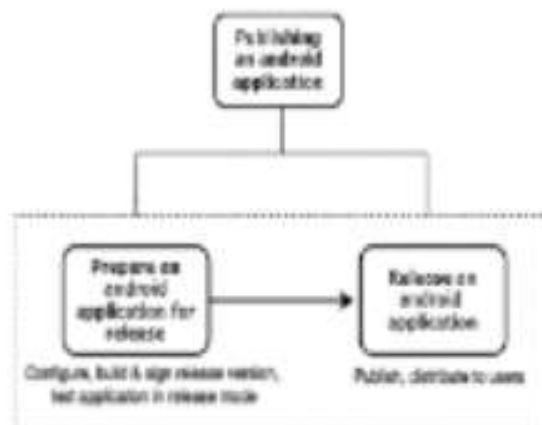
© All rights reserved.



Phases of Publishing Android Applications

The process of publishing consists of 2 phases :

- Preparing the application for release
- Releasing the application to the users



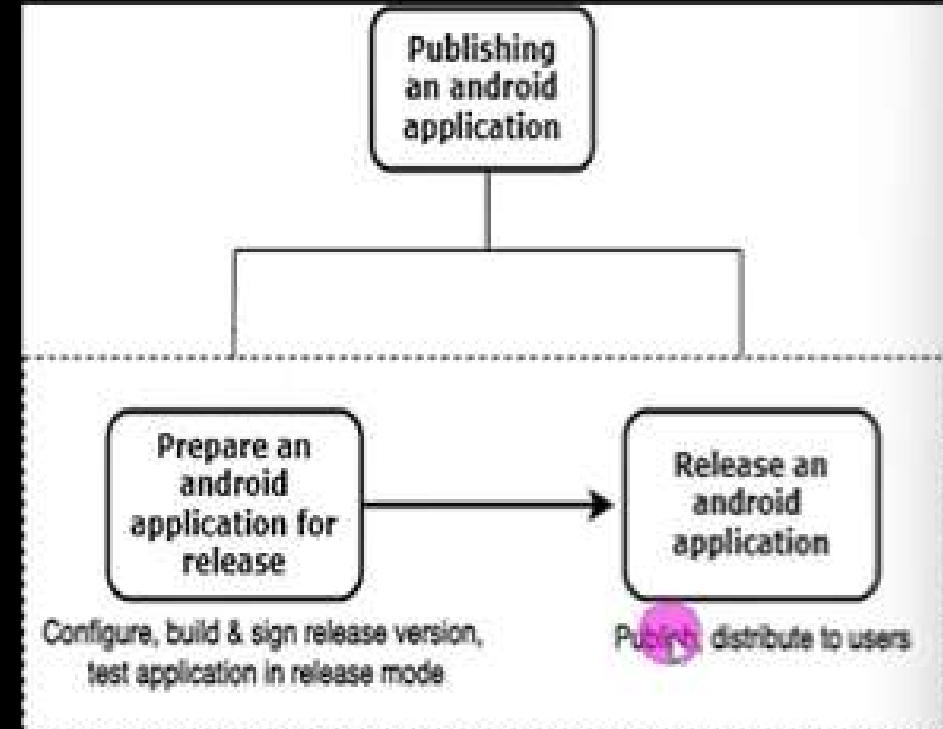
© All rights reserved.

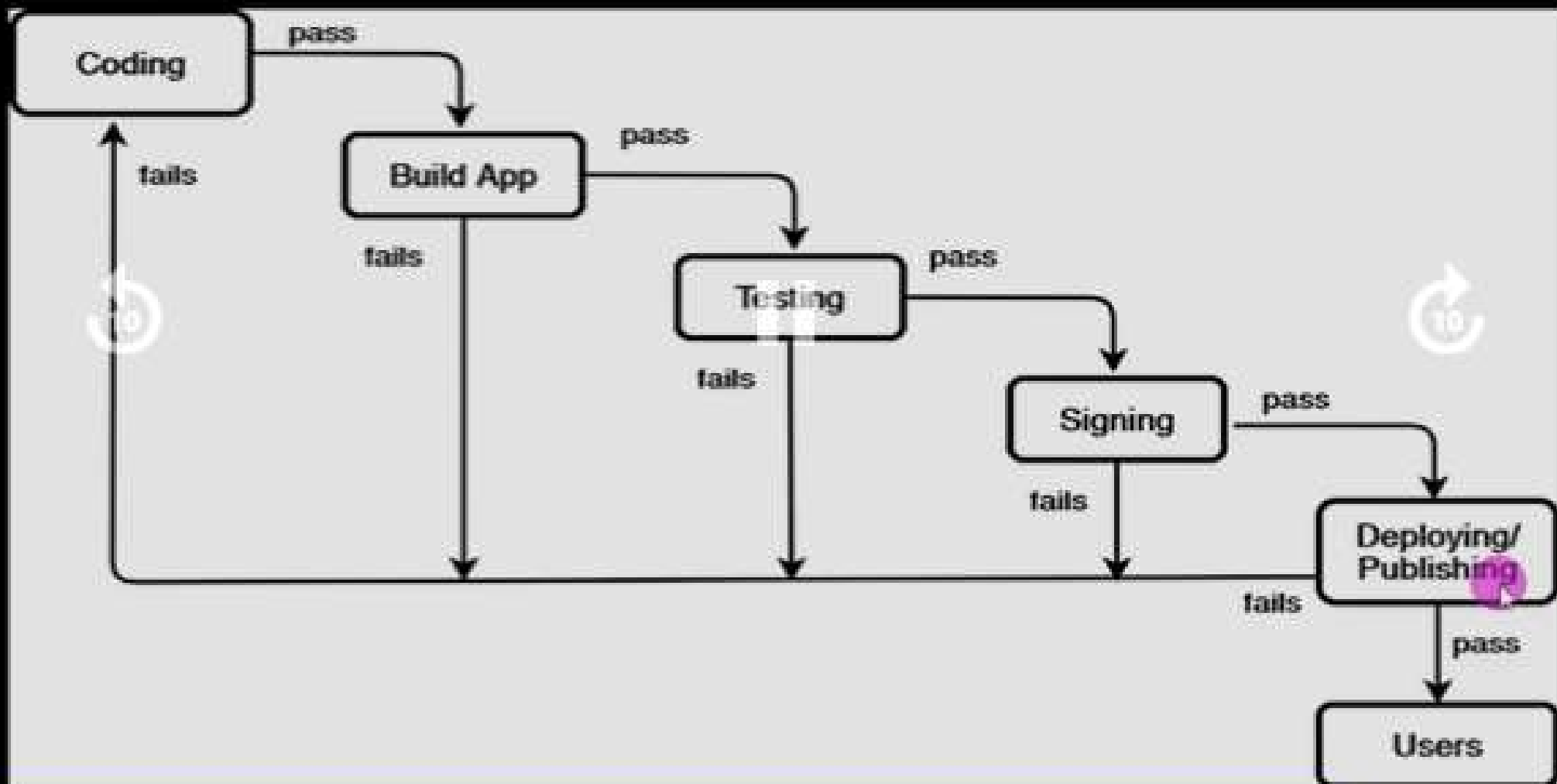


Phases of Publishing Android Applications

The process of publishing consists of 2 phases :

- Preparing the application for release
- Releasing the application to the users





Pre-Release Tasks

1. Specify a suitable name and icon for the application
2. Provide proper versioning for the application
3. Remove unwanted resources, garbage code, and any logging calls used
4. Configure the application with the required permissions in the `AndroidManifest.xml` file
5. Update all the application resources like graphics and multimedia files



Pre-Release Tasks

6. Prepare remote servers used by the application if any
7. Generate a self-signed certificate, digitally sign the application and build an apk
8. Test the release version

© All Rights Reserved



Naming the Application

1. Specify a suitable name and icon for the application
 - Create and use an icon for the application, so that the users can identify and launch the application easily.
 - It helps to manage the application once it is distributed.
 - Choose a suitable name for the application before distribution because the name cannot be changed once it is distributed.

© All Rights Reserved



Naming the Application

- Following changes have to be made in the AndroidManifest.xml file by setting the values of android:icon and android:label attributes in the <application> element

```
<application>
    android:icon="@mipmap/xxx" // specify an icon
    android:label="@string/app_name"
                        // application name
    . . . .
    . . . .
</application>
```

Versioning

2. Provide proper versioning for the application

Versioning:

- It plays an important role in the application development life cycle.
- It helps users to identify which version of the release is installed on their device and upgrade if new release versions are available
- It helps to determine the compatibility and dependencies with other applications

© All Rights Reserved



Versioning

- The version information can be used even by publishing services to perform the tasks depending upon the release version
- There are 2 attributes used to set the application version
 - versionCode
 - versionName

These values are set in the AndroidManifest.xml file directly or in the gradle build file which is then merged during the process of build in the manifest file.

© All Rights Reserved



Versioning - Version Code

versionCode:

- It is a positive integer value used only by the android system for internal purposes and is not shown/known to the users.
- The higher value indicates the latest version.
- This number is used by the android system to prevent users from installing apk with a lower number than the existing one on the device.
- For e.g. android:versionCode = "1"

© All Rights Reserved .NET



Versioning – Version Name

versionName: It represents the applications release version

- It is a string value given in the format "<major>.<minor>.<point>".

For e.g. android:versionName = "1.1.0"

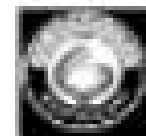
- where the major value is 1, the minor value is 1, and the point value is 0. The value of each of these is incremented by 1 based on the changes made.

© All Rights Reserved .NET



Versioning – Version Name

- <major> value is changed when a new feature/module is added
- <minor> value is changed when some small features are added which were rolled out during development
- <point> value is changed when some bugs are fixed and were not noticed earlier.



Handle Unwanted resources, Garbage code and log calls

3. Remove unwanted resources, garbage code, and any logging calls used

- Any log calls used in the application should be removed.
- The debuggable attribute in the AndroidManifest.xml file should be set to false `android:debuggable = "false"`.
- Any unwanted code, display messages, and resources used for testing flow and checking the proper functioning of the application should be removed.

Dr. K. Subramanian



Configure App with the Permissions

4. Configure the application with the required permissions in the AndroidManifest.xml file

- The permissions in the AndroidManifest.xml file should be verified.
 - To send SMS through the application check whether the permission is set

```
<uses-permission android:name="android.permission.SEND_SMS"/>
```

Dr. K. Subramanian



Configure App with the Permissions

4. Configure the application with the required permissions in the **AndroidManifest.xml** file

- The permissions in the AndroidManifest.xml file should be verified.
 - To send SMS through the application check whether the permission is set

```
<uses-permission android:name="android.permission.SEND_SMS"/>
```

Configure App with the Permissions

- To read/write data stored on external storage verify

```
<uses-permission  
android:name="android.permission.READ_EXTERNAL_STORAGE"/>  
<uses-permission  
android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
```

Set API levels in Manifest file

The API levels used in the application are set by using attributes in `<uses-sdk>` element

- `android:minSdkVersion`: It is used to specify the minimum level of API on which the android application can run. The default value is 1.
- `android:maxSdkVersion`: It is used to specify the maximum level of API on which the android application can run. It is an optional attribute.



Set API levels in Manifest file

- `android:targetSdkVersion`: It is used to specify the **level of API** on which the android application is designed to run.

```
<uses-sdk  
    android:minSdkVersion="14"  
    android:targetSdkVersion="19" />
```

Update Application Resources

5. Update all the application resources like graphics and multimedia files

- Update the required **graphics** and **image/icon files** in the **drawable folder** and **multimedia files** such as audio, and video used by the application

Prepare Remote Servers

6. Prepare remote servers used by the application if any
 - If the android application is using any **database** stored on **remote servers**, check the **connectivity** and ensure the **availability of the network** so that the application works properly without any network issues.

Build an apk

7. Generate a self-signed certificate, digitally sign the application and build an apk

- Any android application has to be digitally signed by the developer before it is released in the market.
- All android applications require a digital certificate and a private key of that certificate to install/upload the application in Google play.
- Google does not appoint any authority to sign the certificates, they are self-signed by the developers.

Dr. R. Subramanian



Build an apk

- The certificate is used to identify the developer and also share data with other applications that are signed with the same certificate.
- The same key is used by the developers to publish different or multiple applications. New versions/releases of the applications also use the same certificate for release in the market.
- Users on the other hand have to fully install the upgraded version if a different certificate is used. Android applications that are not signed are rejected by the emulator or any device.

Dr. R. Subramanian



Modes of Signing Android App

The signing of an android application is done in 2 modes

- Debug
- Release

© All Rights Reserved



Signing an Android App – Debug Mode

- In the android studio, any application that is developed is automatically signed in debug mode when the project is run from the IDE.
- By default, a debug certificate/Keystore with an unknown password and a private key with an unknown password is automatically generated by the android SDK.

© All Rights Reserved



Signing an Android App – Debug Mode

- Whenever the developer makes the changes and runs the project on the emulator or even a real device that is connected using USB to the development machine, it does not ask for a password.
- However, an android application signed in debug mode cannot be distributed to the users.

© All Rights Reserved



Signing an Android App – Release Mode

- To sign a project in release mode, a Keystore and private key are created.
 - Keystore: It is a binary file that consists of a set of private keys. This file has to be kept in a secure and safe folder.
 - Private/Upload key: It is used to identify the author/developer of the application

Signing an Android App – Release Mode

- There are 2 ways for generating a signed apk for an android application in android studio
 - On the menu bar, select build and then use Generate signed APK wizard and provide the required information
 - Configure the project to automatically release the signed APK during the build process by providing the required information in the Open Module setting, an option available in the project browser on right click of the app

© All Rights Reserved

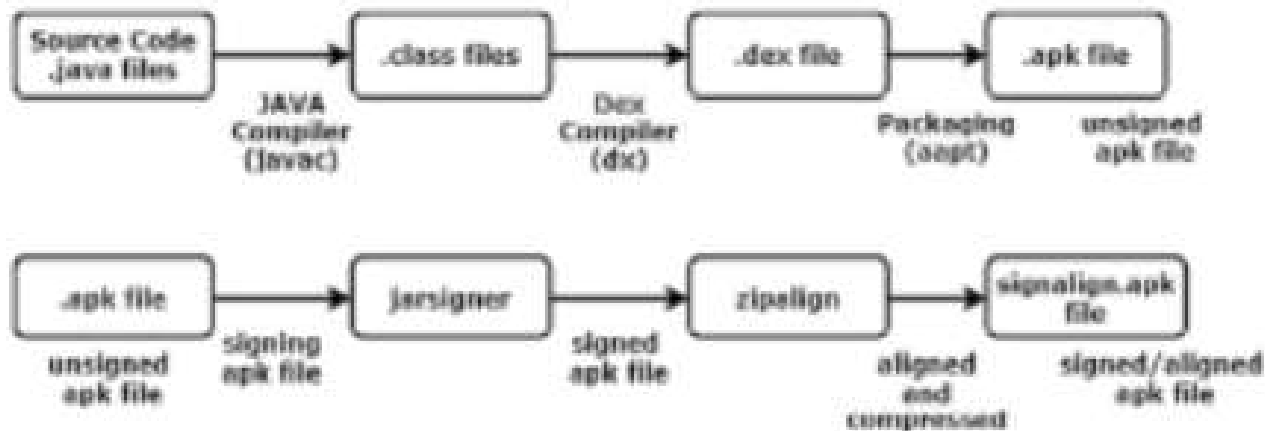


Signing an Android App – Release Mode

- Standard command line tools are also available which are used to generate keys and sign an android application
- For e.g.,
 - keytool is used to generate keys for an android application
 - jarsigner is used to sign an android application
 - zipalign is used to optimize the .apk file

Process of Generating apk file

Process of converting java source file to signed and aligned .apk file



© All Rights Reserved



Test the Release Version

8. Test the release version

- Test the release version of the application on real devices such as handsets, tablets, etc. to ensure that the user interface is user-friendly and working properly on all devices.

Release Application to the Users

Dr. K. K. Srinivasan

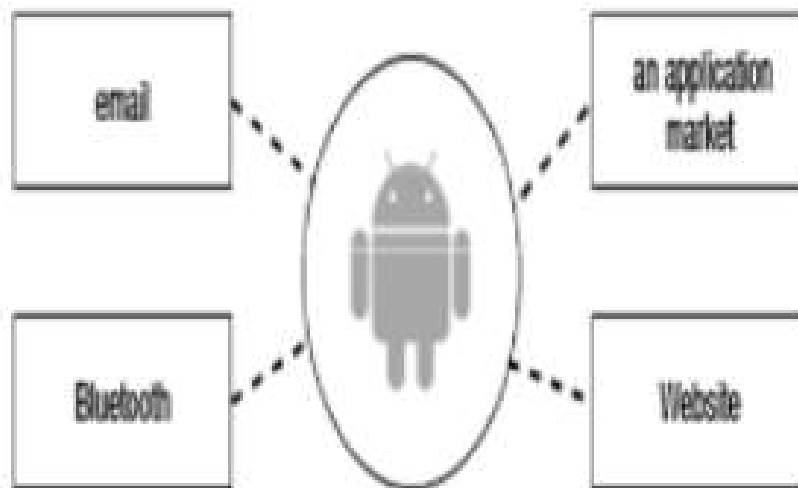


Different ways of Releasing Android Application

- Once the android application is prepared for release and tested it is ready to be distributed to the users
- There are several ways by which the android application can be released
 - Email
 - Bluetooth
 - Website
 - Application Market

Dr. K. K. Srinivasan





Dr. K. K. S. S. S. S.



Different ways of Releasing Android Application

- **Email:** It is one of the simplest and fastest ways to publish an android application. Email the .apk file to the user. But it is unsafe as anyone can pirate and use it.
- **Download from a Website:** Host the android application for release on a website so that the user can download and install it. But this is also unsafe as it's difficult to keep a track of the users who are downloading and using it.

Dr. K. K. S. S. S. S.



Different ways of Releasing Android Application

- **Bluetooth:** .apk file can also be transferred to a mobile device by using Bluetooth by pairing it with the device. Only authorized users can be paired and allowed for file transfer.
- **Google Play:** It is an official marketplace for distributing android applications. Google Play supports publicity, selling, and distribution of android applications not only locally but throughout the world.

© All Rights Reserved



Steps to Release App in Google Play

- Create a Google Developer Account by paying registration fees and signing the terms and agreement policies.
- Product description, feedback questionnaire for content rating, and pricing information for the application are provided through the wizards before uploading the .apk files.
- For uploading the app if the app size is more than 100 MB which is the limitation, APK expansion files are used.

© All Rights Reserved



Steps to Release App in Google Play

- The app can be published by either using alpha, beta, or production options.
- Alpha version [closed testing] is available for testing only for a specific number of invited testers.
- Any interested users, who have been given permission can test the Beta version and send feedback,
- Production release is available to all users of the play store in the countries chosen.

